Analyzing Website Data Collection Efficiency and Network Behavior Using Multiple Web Scraping Techniques

Web scraping, network analysis and machine learning on Premier League 2023-2024 data

Jana Alamir and **Rim Zoghbi**

November 2026

1. Introduction

The goal of this project is to collect the **same data with three different web scraping methods**, compare how efficient they are and how they behave on the network, and then use the collected data to make **predictions with machine learning**.

The three methods are:

- **API**: asking the website's official JSON API for the data,
- **BeautifulSoup**: downloading the HTML page with *requests* and extracting the data from it,
- **Selenium**: opening the page in a real (headless) Chrome browser and reading it.

For each method we measured the time, the number of requests, the downloaded data and the latency, and we observed the network traffic with *ss, iftop, nload, tcpdump* and *Wireshark*. We also tested a firewall rule with *ufw* and a multithreaded version of the API scraper.

Website choice

We chose **TheSportsDB** (www.thesportsdb.com), a free sports database that offers both a website and a JSON API with a free test key. This is important for our project because we can scrape **exactly the same data** from the API and from the website, so the comparison is fair.

We used the **English Premier League, season 2023-2024**: 20 teams, 38 rounds, **380 matches**. Football results are also a good dataset for machine learning (predicting goals and results).

2. Data collected

Every scraper saves the matches in a CSV file with the same 7 columns:

Column	Meaning	Example
date	date of the match	2023-08-11
round	round number (1-38)	1
home_team / away_team	the two teams	Burnley / Manchester City
home_goals / away_goals	final score	0 / 3
result	result for the HOME team: W, D or L	L

All methods collected **380 matches**, and we checked that the files are **exactly identical** (same dates, teams, scores and results). In the season there were 46% home wins, 22% draws and 32% away wins, and an average of 3.28 goals per match.

3. The three scrapers

All scrapers use a shared helper file, **metrics.py**, which starts a timer, counts every request, adds up the size of every response (bandwidth) and measures the time of each request (latency). At the end it adds one row to **data/metrics.csv**. It also makes sure that a failed request never crashes the program (try/except, timeout of 30 s, retries).

3.1 API scraper (scraper_api.py)

The API returns JSON data. The endpoint for a whole season only returns 15 matches with the free key, so we ask for **one round at a time** (eventsround.php): 38 requests of 10 matches. From the ~30 fields of each match we keep only our 7 columns. Between two requests the scraper waits 4 seconds (rate limiting), and if the server answers *429 Too Many Requests*, it waits the number of seconds given in the *Retry-After* header before trying again.

3.2 Multithreaded API scraper (scraper_api_threaded.py)

The same API requests, but made by **4 threads at the same time**. A shared rate limiter gives each thread a time slot so that all threads together send at most one request every 2 seconds (30 per minute, the documented limit). Each thread has its own HTTP session, and the counters in metrics.py are protected with a lock.

3.3 BeautifulSoup scraper (scraper_bs4.py)

The season page of the website normally shows 100 matches, but with **?all=1** in the URL it shows all 380 matches in **one plain HTML table**, without JavaScript. The scraper downloads this page with one request, then BeautifulSoup finds every row (<tr class="season-event-row">) and reads the 5 cells: date, round, home team, score, away team.

3.4 Selenium scraper (scraper_selenium.py)

Selenium starts a headless Chrome browser (no window), opens the same page, waits until the table is there, and reads the same cells as BeautifulSoup. Because the browser downloads many files by itself, we ask Chrome for the list of all downloaded files (JavaScript Performance API) to count the requests and bytes.

Responsible scraping

- a clear **User-Agent** that says this is a university project,
- **rate limiting** (4 s between API requests, respect of Retry-After),
- **timeouts and error handling** (no crash on a failed request),
- one reused connection (*requests.Session*) instead of a new one for every request.

4. Problems we faced and how we solved them

Problem	Solution
The API's whole-season endpoint returns only 15 matches with the free key.	Ask one round at a time: 38 requests, 380 matches.
With 2.5 s between requests, the API answered *429 Too Many Requests* after ~11 requests; our first run got only 210 of 380 matches. Our retries (5 s) came before the 6 s the server asked for.	4 s between requests and wait the *Retry-After* time after a 429. Result: 380 of 380 matches.
The website season page shows only 100 matches.	Found that **?all=1** shows all 380 matches in one page.
Each team name appears twice in the HTML (full name for computers, short name like 'Manchester Ci' for phones).	Take only the link with the CSS class *d-lg-block* (desktop version).
Selenium only reads *visible* text, and the full names and the round column are hidden on small screens.	Open the browser with a large window (1920x1080).
The browser cannot tell us the size of files from other domains (images, Google ads): they count as 0.	We use the tcpdump capture to measure the real traffic.
Our measurement script hung after the API run (bash *wait* waited for a background loop that never ends).	Wait only for the tcpdump process; allow running only some steps again.

5. Network results

5.1 Performance metrics

Method	Total time (s)	Requests	Data (KB)	Avg latency (ms)	Matches
API	516.57	41	668.3	134.8	380
API threaded	413.53	50	730.8	109.4	380
BeautifulSoup	0.53	1	1,040.3	237.9	380
Selenium	37.66	60	1,686.4	293.0	380

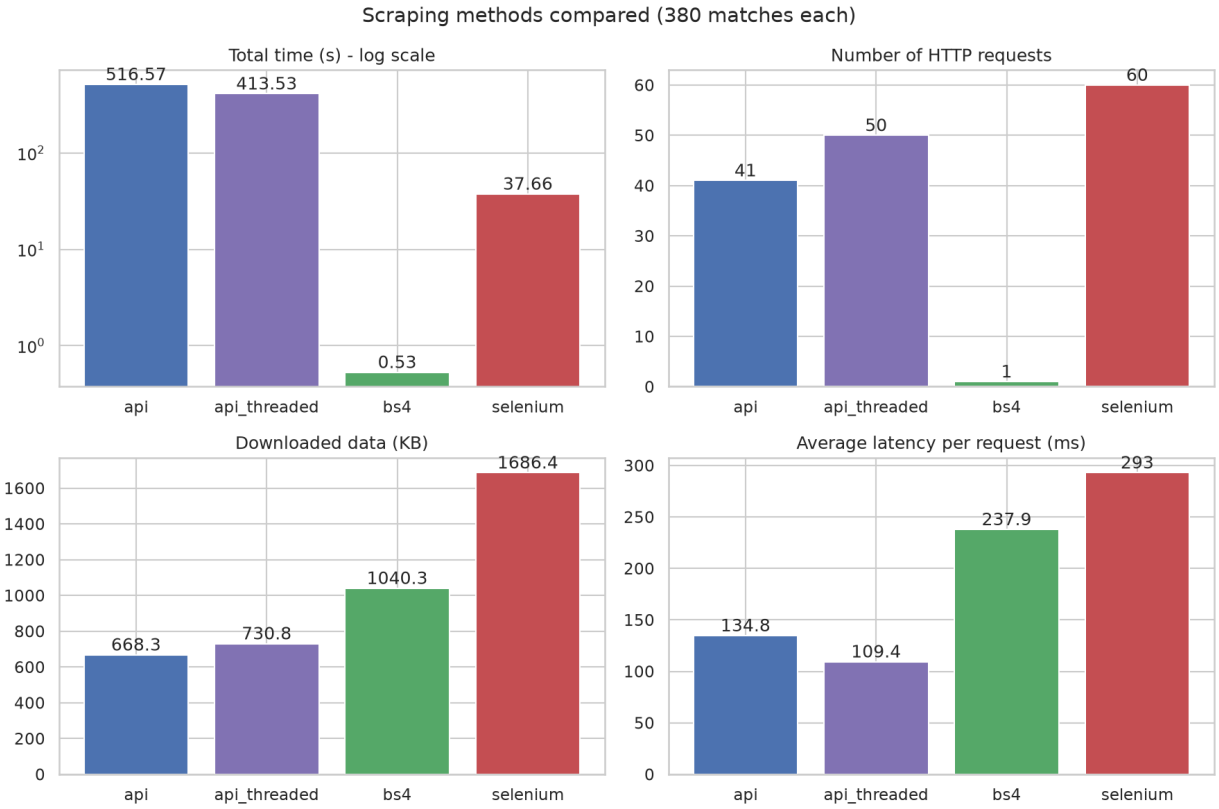


Figure 1: comparison of the four scrapers (time is on a log scale).

- **BeautifulSoup is the fastest** (0.53 s): one request is enough because the website shows the whole season in one page.
- **Selenium** needs 38 s: starting the browser, loading all the page's files, and reading 380 rows through the WebDriver (each cell is a separate command).
- **The API is the slowest in total** (517 s), but not because the API is slow: the time spent inside the requests is only about 5.5 s. About 99% of the time is **waiting** because of the rate limit of the free key and the 38 separate requests.
- Per request, the API is the fastest (135 ms against 238 ms for the HTML page) and it downloads the least data (668 KB): only the JSON data we need.
- The **threaded API** scraper was about 20% faster (414 s), but needed 50 requests because it got 12 *429* errors (the normal API scraper: 3).

5.2 Packet captures (tcpdump)

During each run we captured all TCP traffic on ports 80 and 443 with tcpdump and analyzed the .pcap files with Python (scapy). Even with HTTPS, the first message of every connection (the TLS *ClientHello*) contains the server name in clear text (SNI), so we can see **which domains** each method contacted.

Method	Packets	Bytes on network (KB)	TCP connections	Domains	Duration (s)
API	381	184.0	1	1	516.6
BeautifulSoup	38	35.7	1	1	0.5
Selenium	4297	7,934.6	24	12	37.5

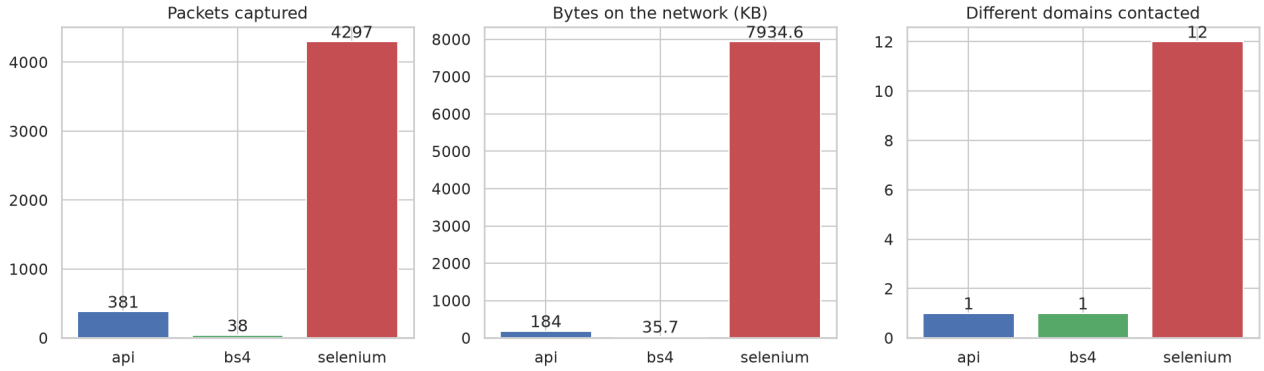


Figure 2: what tcpdump captured for each method.

The API and BeautifulSoup use **only one TCP connection** to www.thesportsdb.com. Selenium opened **24 connections** to **12 different domains**.

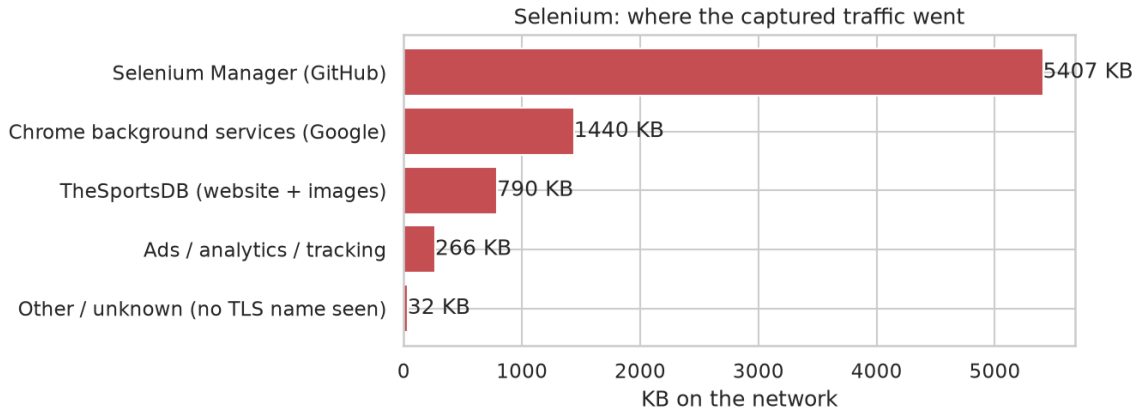


Figure 3: where Selenium's captured traffic went.

Only 790 KB of the 7934 KB (10%) captured during the Selenium run came from TheSportsDB. Most of the traffic was **Selenium Manager** (it downloads version lists from GitHub to choose the right ChromeDriver, at every start) and **Chrome's background services** (updates, Google account, optimization guide). The page also loads **ads and trackers** (Google Tag Manager, Cloudflare Insights, Plausible). Note: Chrome also uses QUIC (HTTP/3 over UDP), which our TCP-only capture did not include, so the real traffic was even bigger.

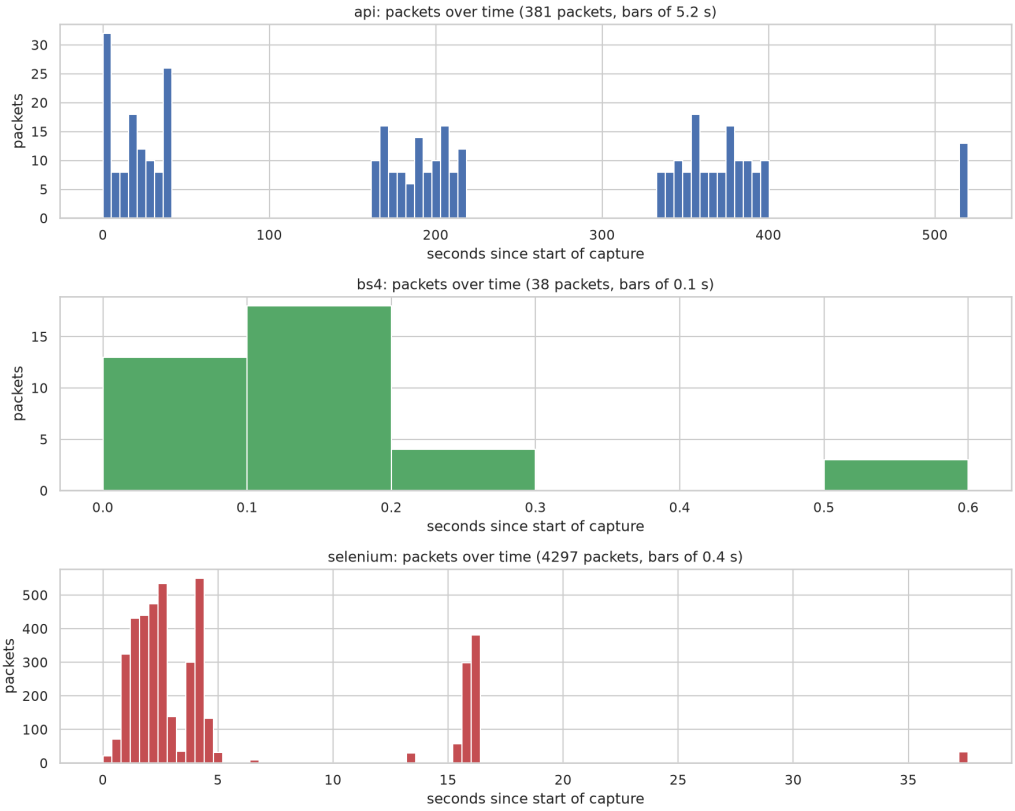


Figure 4: packets over time. API: small regular bursts (one per round); BeautifulSoup: one short burst; Selenium: one big burst then background traffic.

5.3 Live monitoring (screenshots)

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/ss_api.png** and run *python report/build_report.py* again.

ss during the API run: a single HTTPS connection (python to TheSportsDB).

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/ss_selenium.png** and run *python report/build_report.py* again.

ss during the Selenium run: many parallel connections from Chrome (TCP and QUIC/UDP on port 443).

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/nload_api.png** and run *python report/build_report.py* again.

nload during the API run: small, regular traffic peaks.

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/nload_selenium.png** and run *python report/build_report.py* again.

nload during the Selenium run: a much bigger peak.

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/iftop_api.png** and run *python report/build_report.py* again.

iftop during the API run: only one remote host.

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/iftop_selenium.png** and run *python report/build_report.py* again.

iftop during the Selenium run: many remote hosts at the same time.

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/tcpdump.png** and run *python report/build_report.py* again.

tcpdump: the TCP three-way handshake (SYN, SYN-ACK, ACK) and then encrypted TLS traffic.

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/wireshark_conversations.png** and run *python report/build_report.py* again.

Wireshark - Statistics > Conversations (Selenium capture).

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/wireshark_protocols.png** and run *python report/build_report.py* again.

Wireshark - Statistics > Protocol Hierarchy: almost all data is TLS (encrypted HTTPS).

6. Security part

6.1 Firewall test with ufw

To show how a firewall can control scraping traffic, we blocked all outgoing HTTPS traffic with **sudo ufw reject out 443/tcp** and ran the API scraper (2 test rounds). We used *reject* instead of *deny* so that the connection is refused immediately (with *deny*, packets are silently dropped and every request waits for its 30 s timeout).

	Rule active (443 blocked)	Rule removed
Result	every request failed: 'Network is unreachable'	requests work normally
Matches collected	0	20
Time	44.5 s (5 attempts x 2 rounds, waiting between retries)	4.5 s
Did the program crash?	No: errors are caught, it retries and continues	-

[SCREENSHOT PLACEHOLDER]
Save the screenshot as **network/screenshots/ufw.png** and run *python report/build_report.py* again.

ufw status with the REJECT OUT rule, and the failed API run.

6.2 Rate limiting

Rate limiting protects the server, and also protects us: when we went too fast, the server blocked us (429 errors) and we lost data. With 4 s between requests and respecting the *Retry-After* header, we got all 380 matches. The real limit of the free key was stricter than the documented ~30 requests per minute, so rate limits must be **tested, not assumed**.

6.3 Multithreading: speed vs. stability

With 4 threads the API scraper was 20% faster, but it received 12 *429* errors instead of 3. When the server limits the number of requests, threads mainly make us hit the limit faster. They give a small gain in speed but less stability and less politeness towards the server.

7. Machine learning results

7.1 Method

- Matches sorted by date; no missing values and no duplicates.
- Features calculated **only from previous matches** (no data leakage), for the home team and the away team: **form** (points in the last 5 matches), **average goals scored** and **average goals conceded** in the last 5 matches. The first 3 features describe the home team and the last 3 the away team, so the model can learn the home advantage.
- Matches where a team had fewer than 5 previous games were skipped: 329 matches remain.
- **Training** on the 229 earlier matches, **test** on the **last 100 matches** of the season (all 20 teams).

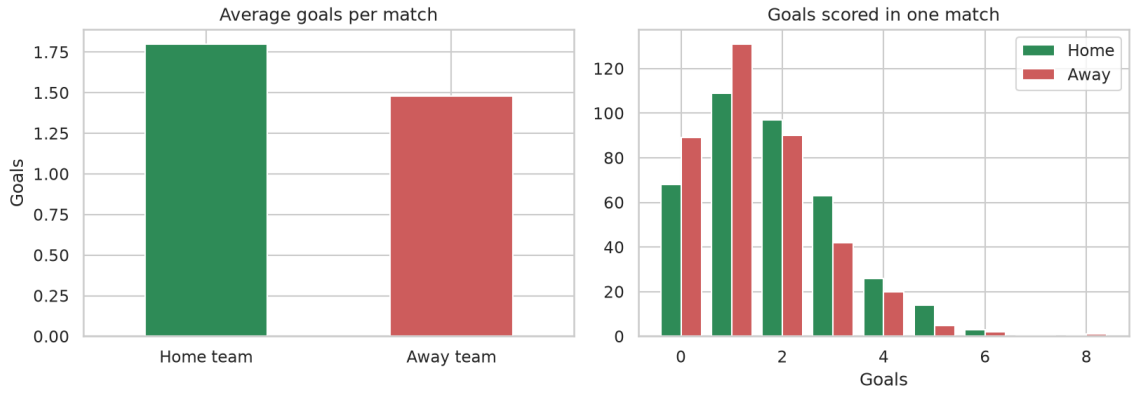


Figure 5: home teams score more goals than away teams (home advantage).

7.2 Regression: predicting goals (MAE)

Model	MAE home goals	MAE away goals	Average MAE
Linear Regression	1.09	0.94	1.01
Random Forest	1.14	0.97	1.06
Baseline (average)	1.15	1.04	1.10

Linear Regression is the best model: on average our predicted number of goals is about **1.0 goal** away from the real number. This is only a little better than always predicting the average, because football scores are very random. Only 10 of the 100 exact scores were predicted correctly.

7.3 Classification: predicting W / D / L (accuracy)

Model	Accuracy on the last 100 matches
Logistic Regression	59%
Random Forest	54%
Baseline (always home win)	46%

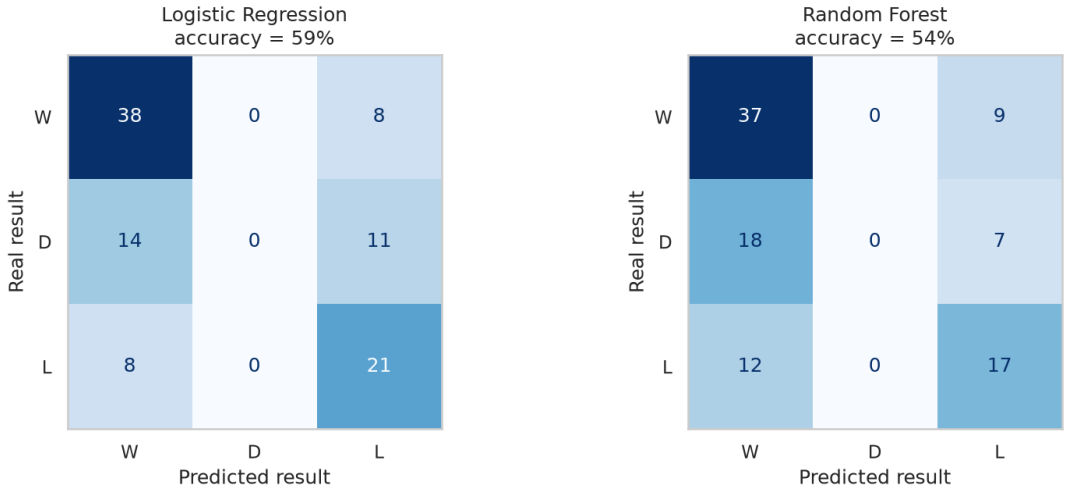


Figure 6: confusion matrices (rows = real result, columns = predicted result).

Logistic Regression reached **59%** accuracy, against **46%** for the simple baseline that always predicts a home win (+13 percentage points). So the form features really help. The models are good at home wins and home losses, but **never predict a draw**: a draw looks like 'two similar teams', which can also end in a win or a loss.

7.4 Predictions for the last 100 matches

The full table (home team, away team, predicted score, real score, predicted result, real result, correct or not) is in **data/predictions_last100.csv** and in the ML notebook. Summary: **59 of 100 results correct**, 10 exact scores. The first 10 matches:

Home team	Away team	Pred. score	Real score	Pred.	Real	OK
Fulham	Tottenham	2-2	3-0	W	W	yes
Luton	Nottingham Forest	2-2	1-1	L	D	no
West Ham	Aston Villa	2-1	1-1	W	D	no
Aston Villa	Wolves	2-1	2-0	W	W	yes
Bournemouth	Everton	2-1	2-1	W	W	yes
Brentford	Manchester United	2-2	1-1	L	D	no
Chelsea	Burnley	2-1	2-2	W	D	no
Newcastle	West Ham	2-2	4-3	W	W	yes
Nottingham Forest	Crystal Palace	2-2	1-1	L	D	no
Sheffield United	Fulham	1-2	3-3	L	D	no

Limitations: only one season, only 6 simple features, and a small test set (1 match = 1%). We chose the best model by looking at the test results, which makes the result a little optimistic.

8. Optimization recommendation: which method is best?

	API	BeautifulSoup	Selenium
Network efficiency	best (only data)	good (1 page)	worst (many files, domains)
Total speed (our case)	slow (rate limit)	fastest	medium
Stability	high (official format)	breaks if the HTML changes	most fragile
Data quality	clean JSON fields	needs careful parsing	needs careful parsing
Needed when...	an API exists	data is in plain HTML	page needs JavaScript

Our recommendation:

- Use the **API** when it gives all the data: it is the most efficient for the network, the most stable and gives the cleanest data. With a paid key (whole season in one request) it would also be the fastest.
- Use **BeautifulSoup** when the data is in plain HTML and the API is limited. On TheSportsDB it was the best compromise: 1 request, under 1 second, same data.
- Use **Selenium only when the page needs JavaScript**: a full browser downloads a lot of traffic that has nothing to do with the data (Selenium Manager, Chrome services, ads, trackers).
- Always respect the rate limit; adding threads does not help when the server limits the request rate.

9. Conclusion

We collected the same 380 Premier League matches with three scraping methods and got identical data from all of them. The methods are very different on the network: the API sends only the data, BeautifulSoup needs one HTML page, and Selenium generates much more traffic to many servers. In our case the free API key's limits made the API the slowest method in total time, even though it is the most efficient per request. The network tools (ss, iftop, nload, tcpdump, Wireshark) confirmed these differences, and the ufw test showed how a firewall rule can stop scraping traffic completely.

With machine learning, simple features based on the last 5 matches predicted the result of the last 100 matches with 59% accuracy, better than the 46% of always predicting a home win, and predicted goals with an error of about one goal. More seasons and more features would be needed to do better.